



# Azure Virtual Machine Extension Standardisation

Project 24CC3046-P090 — Category: Operations. Cross-team initiative between T090 and T192 to ensure consistent, reliable VM extension state across subscriptions.

OPERATIONS

T090 • T192



# Problem Statement & Key Use Cases



## Problem

VMs sometimes have missing or failed extensions leaving machines partially configured. Multiple agents may conflict and consume CPU, memory and network.



## Use Case – Detection

Detect VMs missing required monitoring or security agents across subscriptions and resource groups.



## Use Case – Remediation

Automated, policy-driven remediation to install or repair approved extensions and verify success.

# Objectives – What Success Looks Like

## Discover

Automatically find all Azure VMs using Resource Graph + tags.

## Inspect

Enumerate installed extensions, versions, and statuses.

## Check

Compare installed extensions against the standard compliance baseline.

## Detect

Flag missing, failed, or conflicting agents for action.

## Remediate

Apply approved extensions via automation runbooks or Azure SDK.

## Verify & Report

Confirm state post-remediation and generate compliance reports.

**Main flow:** Discover → Inspect → Check → Detect → Remediate → Verify → Report

# Proposed System Architecture



Centralised compliance engine driven by discovery feeds, inspection agents (read-only + remediation), and monitoring pipelines feeding dashboards and alerts.

# Azure Requirements & Supporting Services



Core services required:

- Azure VMs — target machines for standardisation
- Virtual Network & Subnets — ensure reachability for agents
- VM Extensions — the objects being standardised
- Azure Monitor Agent + Log Analytics — monitoring & evidence
- Resource Graph — fast VM/resource discovery
- Python + Azure SDK — automation and orchestration

Optional supporting services: Azure Automation, Defender for Cloud, Key Vault (secrets), Storage accounts for artefacts.

# Functional Requirements (FR)

## 1 FR-01 Discover VMs

Scheduled and on-demand scans via Resource Graph.

## 2 FR-02 Inspect Extensions

Collect extension name, version, publisher, status and timestamp.

## 3 FR-03 Check Compliance

Compare against an approved extension manifest per workload type.

## 4 FR-04 Detect Missing/Failed

Generate tickets/alerts for non-compliant VMs.

## 5 FR-05 Remediate

Automated install/repair using managed identity or runbooks.

## 6 FR-06 Verify

Re-inspect post-remediation to confirm healthy extension state.

## 7 FR-07 Report

Compliance reports per subscription, RG, tag and team.

## 8 FR-08 Dashboard

Visual summary: compliance %, failing extensions, remediation backlog.

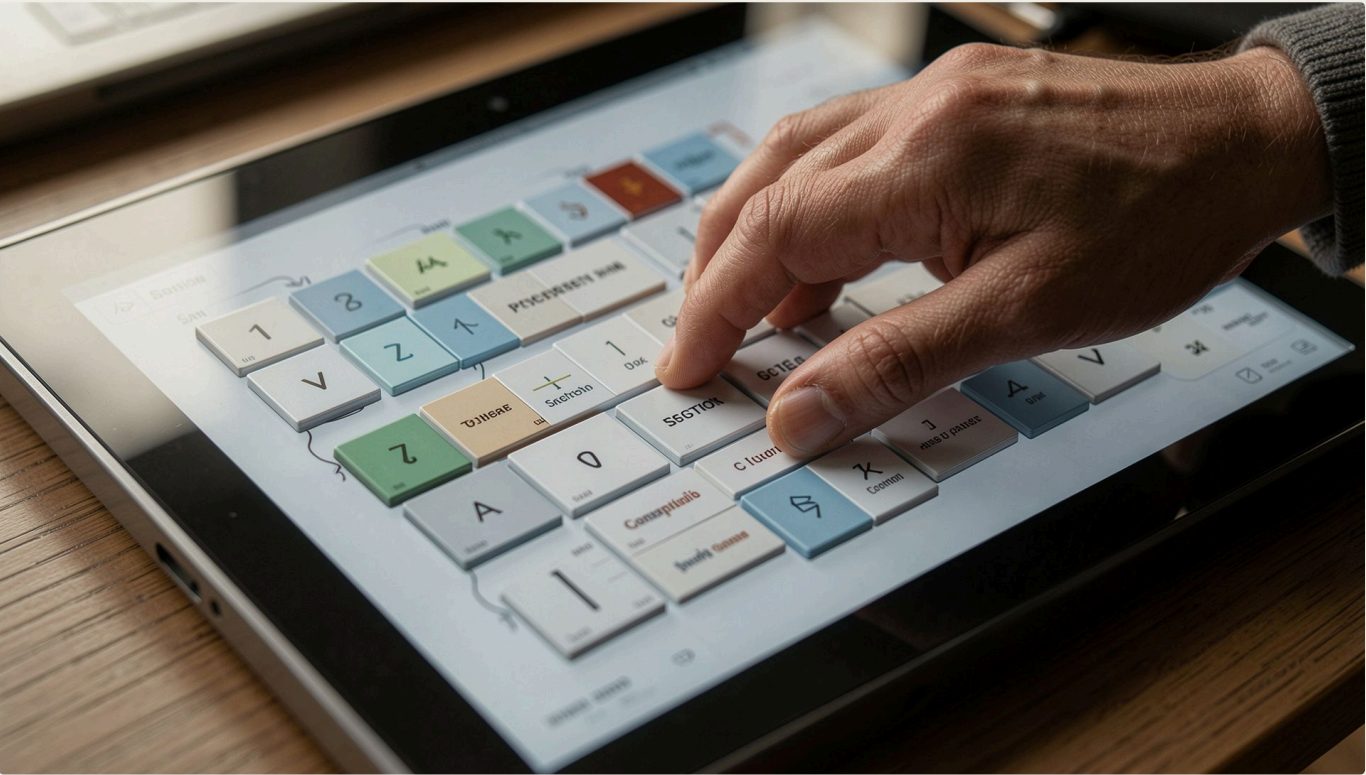
## 9 FR-09 Alerts

Actionable alerts to teams with remediation links.

## 10 FR-10 Scheduled Scans

Configurable cadence with throttling and delta-only scans.

# Project Modules & Responsibilities



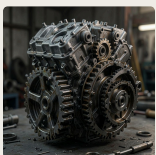
## M1 – VM Discovery

Resource Graph queries, tag-based filters, scheduler service.



## M2 – Extension Inspection

Gather extension inventory and health metadata.



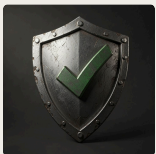
## M3 – Compliance Engine

Ruleset evaluation, baseline manifests and policy definitions.



## M4 – Remediation

Runbooks / SDK actions to apply or fix extensions.



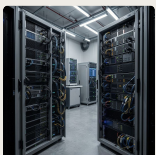
## M5 – Verification

Post-action checks and automated rechecks.



## M6-M9 – Monitoring, Dashboard, Reporting, Alerts

Telemetry ingestion, visualisation, scheduled reports and notifications.



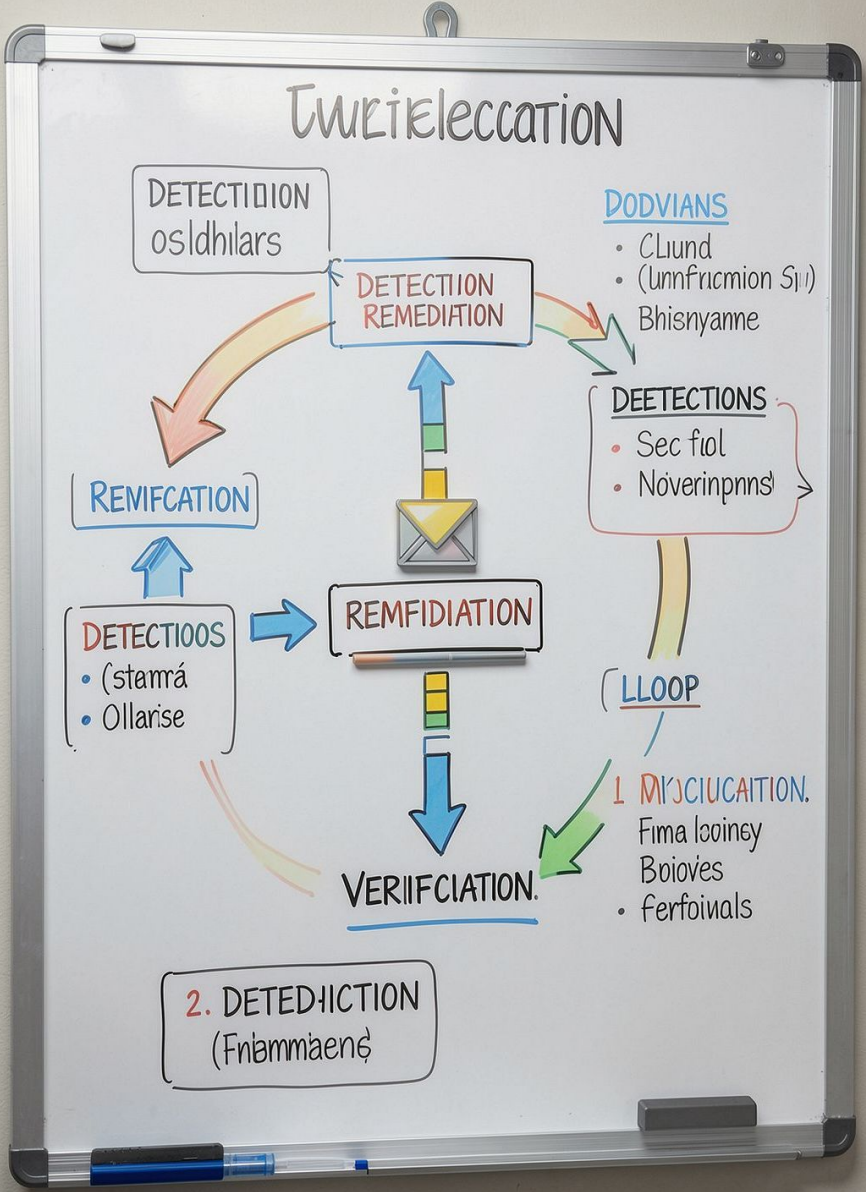
## M10 – Testing

Test harness for failure modes: missing/failed/conflict cases.

# Compliance & Automated Remediation Flow



Built-in safeguards: dry-run mode, approval gates for remediation, retry with exponential back-off, and detailed audit logs for every action.



# Risks, Cost Controls & Low-cost Alternatives

## Top Risks

- Extension deployment failures
- Conflicting agents causing instability
- Insufficient RBAC/permission errors
- Network egress/IMDS restrictions
- Unexpected operational costs

## Controls & Cost Options

- Retry + verification and rollback on failure
- Least-privilege managed identities for actions
- Monitor cost with budgets and alerting
- Use small test VMs, student subscriptions or limited Log Analytics tiers
- Option: Managed Identity instead of Key Vault for simple secrets

# Testing Plan, Deliverables & Next Steps

## Testing

Run scenarios: compliant VM, missing extension, failed extension, remediation success/failure, verification step.

## Deliverables

Azure test environment, Python source code, compliance/remediation module, scheduled reports, test evidence, GitHub repo and documentation.

## Next Steps

1. Finalise extension baseline and approval list
2. Build discovery + inspection PoC
3. Implement remediation runbooks and verification
4. Deploy dashboard and schedule reports

📌 Recommendation: start with a limited-scope pilot (1–2 subscriptions, 10–20 VMs) to validate safety and metrics before broad rollout.

